

Topic 26 — Structured Outputs with Few-Shot Prompting

Full Stack Python + AI Roadmap • Phase 3: Advanced Prompt Engineering • 3 layers: New → Intermediate → Expert



Layer 1 — New to the Concept

A **structured output** is model output in a strict, machine-readable format — almost always **JSON** — instead of free-flowing prose.

An LLM's default is conversational text ("Sure! The sentiment is positive, and I'd rate the urgency as fairly high..."). Your *code* can't use that — it needs `{"sentiment": "positive", "urgency": "high"}`, clean fields it can parse and act on.

Structured output prompting is the craft of reliably getting that clean shape, and **few-shot** is one of the best tools for it: showing 2–3 examples of the exact JSON you want beats merely *describing* it. Analogy: the difference between asking "tell me about this candidate" (a paragraph) and handing over a **form to fill in** (structured fields). Structured output = giving the LLM a form.



Layer 2 — Few-Shot for Reliable JSON

Describing the format is okay; *showing* it with examples is better:

```
prompt = """Extract product info as JSON with keys: name, price, in_stock.
```

```
Text: "The UltraBook Pro is available now for ₹95,000"
```

```
Output: {"name": "UltraBook Pro", "price": 95000, "in_stock": true}
```

```
Text: "Sorry, the AirMax 5 (₹12,999) is currently sold out"
```

```
Output: {"name": "AirMax 5", "price": 12999, "in_stock": false}
```

```
Text: "Grab the SoundBud X for just ₹2,499 – plenty in stock!"
```

```
Output: """
```

Two examples teach at once what a description struggles to convey: **exact key names** (`name` , `price` , `in_stock`); **types** (`price` is a number — no ₹, no commas; `in_stock` is a real boolean); and **edge handling** (how "sold out" maps to `false`).

```
import json
```

```
response = call_llm(prompt, temperature=0) # determinism (Topic 24) helps here
```

```
data = json.loads(response)
```

```
print(data["price"]) # 2499 → clean integer your code can use
```

i Natural pairing: structured output + `temperature=0` — consistent shape every time.



Layer 3 — Expert (making it bulletproof — Pydantic tie-in)

1. Validate with Pydantic — the output guardrail

```
from pydantic import BaseModel

class Product(BaseModel):
    name: str
    price: int
    in_stock: bool

response = call_llm(prompt, temperature=0)
product = Product.model_validate_json(response)  # parses + validates in one step
# malformed field → RAISES → you catch bad output
print(product.price)  # guaranteed to be an int
```

✓ **Guardrails made concrete:** the LLM proposes, Pydantic disposes. Malformed output is caught immediately instead of corrupting downstream data. (Recall Topics 19 & 22.)

2. The modern way — native structured output / function calling

```
# OpenAI structured outputs – pass the Pydantic model directly
response = client.beta.chat.completions.parse(
    model="gpt-4o",
    messages=messages,
    response_format=Product,  # ← the model MUST return this schema
)
product = response.choices[0].message.parsed  # already a validated Product
```

Current best practice: the API *guarantees* schema-valid JSON (no parsing errors); you often still provide few-shot examples to guide *content* quality within the schema. LangChain's `.with_structured_output(Product)` wraps exactly this. Prefer it over prompt-only JSON where available.

3. Why few-shot still matters with schema enforcement

Schema enforcement guarantees *shape*, not *good judgment*. Examples still teach: how to handle ambiguous inputs (what maps to `in_stock: false`), domain conventions (strip currency symbols, normalize units), and edge cases (missing data → `null` vs a default). Pro pattern = **both**: schema enforcement for structure + few-shot for content decisions.

4. Common failure modes & fixes (prompt-only JSON)

Problem	Fix
Wraps JSON in markdown fences	Strip fences, or instruct "no markdown"; schema mode avoids this
Adds prose ("Here's the JSON: ...")	"Output ONLY valid JSON, nothing else" + few-shot showing bare JSON
Trailing commas / single quotes → invalid	<code>temperature=0</code> , few-shot, or native schema mode
Inconsistent keys across calls	Few-shot pins exact keys; schema mode enforces them
Hallucinated extra fields	Pydantic strict mode rejects them

5. Nested & complex schemas — examples earn their keep

```
class Order(BaseModel):
    customer: str
    items: list[dict]      # or a nested Item model
    total: float

# one clear nested example teaches the whole structure instantly
```

For nested structures, few-shot examples become *more* valuable — one well-formed nested example communicates the shape better than a paragraph of description. Recall nested Pydantic models from Topic 19.

6. The robust production pipeline

1. Define schema as a Pydantic model (contract)
2. Prompt with 2–3 few-shot examples of it (guide content)
3. Use native structured output if available (guarantee shape)
else `temperature=0` + "JSON only" (maximize consistency)
4. Validate the response with Pydantic (guardrail)
5. On validation failure → retry / repair (resilience)

i The last-mile trick: if validation fails, re-send the error to the model ("your output failed validation with this error, fix it") — a self-correcting loop. LangChain's `OutputFixingParser` automates this.

Interview Q&A Cards

Q: What is structured output and why does it matter?

A: Strict machine-readable output (usually JSON) instead of prose, so code can parse and act on it. Essential whenever an LLM's result feeds into a program rather than a human reader.

Q: Why use few-shot for structured output?

A: Examples reliably teach the exact keys, value types, and edge-case handling — more effectively than describing the format in words, especially for nested schemas.

Q: Prompt-based JSON vs native structured outputs?

A: Prompt-based hopes the model conforms; native structured outputs / function calling constrain the model to a schema, guaranteeing valid shape. Prefer native where available, still use few-shot for content quality.

Q: How do you make structured output bulletproof?

A: Validate every response with Pydantic (an output guardrail), and on failure feed the validation error back to the model to self-correct — a retry/repair loop, e.g. LangChain's OutputFixingParser.

 **Memory hook:** *Structured output = give the LLM a form (JSON), not a conversation. Few-shot shows exact keys/types/edge cases better than describing them. Modern way: native schema enforcement (function calling) guarantees shape. Always validate with Pydantic (the guardrail). Failure → feed the error back to self-correct.*